Database Row Level Security (RLS) Mapping

This document explicitly maps the **Row Level Security (RLS)** configuration and active access policies across the 69 tables in the live Supabase database.

(Note: Row Level Security is Postgres's internal mechanism for determining if a specific logged-in user has permission to `SELECT`, `INSERT`, `UPDATE`, or `DELETE` a specific row in a table.)

🔒 The Global Security Pattern: Open / Trust Architecture

Major Finding: Your Supabase database is fundamentally operating on an **Open / Trust** architecture.

The vast majority of your 69 tables have **Row Level Security entirely disabled.**

Instead of relying on Postgres to block unauthorized reads at the database level, your application relies on:

1. **Frontend Logic:** React UI components executing filtering queries (e.g., `WHERE org_id = '...'`) to only display relevant data.
2. **Server-Side RPCs:** Custom SQL functions executed with `SECURITY DEFINER` privileges that handle their own internal authentication checks (using `auth.uid()`) before making database edits.
3. **Application APIs:** Deno Edge Functions serving as middlemen to authenticate and modify data programmatically.

🛡 Tables with Active RLS Enabled

There is only **one** major, verified custom data table actively utilizing Row Level Security enforcement at the database root level.

Table Name	RLS Status	Policy Intention
`task_submissions`	**ENABLED**	Protects the core gamification mechanism. It strictly dictates that only authorized users or system roles can insert or modify point submissions to prevent cheating or arbitrary score inflation.
(Supabase native)	*(N/A)*	Tables like `auth.users` inherently carry secure, un-editable policies handled directly by Supabase's internal identity platform.

⚠ Known Exceptions & Developer Notes

During the audit of the local SQL source files, specific developer comments were found indicating that RLS features had been tested but intentionally removed or commented out.

Table Name	RLS Notes / Discoveries

Table Name	RLS Notes / Discoveries
`task_steps`	Explicitly discovered `ALTER TABLE task_steps ENABLE ROW LEVEL SECURITY;` commented out (disabled) in the source code. This confirms an intentional design choice by the developers to keep the checklist UI fast and open for read/writes across teams without permission blockers.

🔓 Tables Operating with RLS Disabled (Public Access)

Because the system relies on Frontend filtering and `SECURITY DEFINER` RPCs, **the remaining ~67 custom tables currently have no active RLS policies attached.**

If an authenticated user obtains the direct REST API endpoint for these tables, they could technically query the raw data inside them unless blocked by other application configurations.

(All Standard Application Tables are Open):

- `orgs`, `company_details`
- `profiles`, `departments`, `teams`, `team_members`
- `projects`, `project_members`, `tasks`, `task_steps`
- `attendance`, `leaves`, `time_logs`, `timesheets`
- `employee_finance`, `payroll`, `payslips`
- `conversations`, `messages`, `attachments`, `message_reactions`
- `skills_master`, `employee_skills`, `task_skills`
- `announcements`, `notifications`, `tickets`
- *(And all dormant or unused modules like hiring and invoicing)*

💡 Recommendation for the Future

An Open/Trust architecture is incredibly common (and performant!) for early-stage B2B SaaS applications or internal company tools where every logged-in user is generally trusted.

However, if you ever plan to open this application to external competing clients or strictly multi-tenant environments where a bug in your frontend React code could accidentally display Company A's data to Company B, **you must enable RLS on fundamental tables (specifically `profiles`, `orgs`, and `projects`)** forcing `WHERE org_id = auth.jwt() -> 'user_metadata' -> 'org_id'`.